CÉGEP De Chicoutimi

LIVRABLE 2 : CAHIER DE CONCEPTION

JEU GO - Dév. App. Avancé | 420-D60-CH | Gr. 01

Henri Pitre
2026-04-17

TABLE DES MATIÈRES

TABLE DES MATIÈRES 2
1. PERSPECTIVE GLOBALE 4
1.1 Rappel de fonction 4
1.2 Contraintes 4
1.3 Définition de l'architecture de l'application 4
2. MODELISATION ORIENTEE OBJET 5
2.1 Encapsulation 5
2.2 Collections et listes d'objets 5
2.3 Héritage 5
2.4 Rôles par classe 5
2.5 Classes MODEL 6
2.6 Diagramme de classes 8
2.7 Diagrammes de cas d'utilisation 9
2.7.1 Démarrer la partie 9
2.7.2 Jouer un coup 9
2.7.3 Passer son tour 10
2.7.4 Abandonner la partie 10
2.7.5 Terminer la partie 11
2.8 Validation et rappel de contraintes importantes 11
4. STRATÉGIE MATHÉMATIQUE 12
4.1 Détéction des libertés 12
4.1.1 Algorithme de vérification 12
4.2 Pseudo-code 13
5. EXIGENCES DE CONCEPTION 13
6. CONCEPTION DE L'INTERFACE 14
6.1 Principes de conception 14
6.2 Architecture de l'interface 14
6.3 Maquettes fonctionnelles 15
6.3.1 Interface de création de partie 15
6.3.2 Interface de jeu en cours 16
6.3.3 Interface de résultats 17

6.4 Responsivité et redimensionnement 17
7. STRATÉGIES DE TEST 18
7.1 Philosophie de test 18
7.2 Tests unitaires 18
7.2.1 Objectifs 18
7.2.2 Éléments testés 18
7.3 Tests d'intégration 18
7.4 Tests fonctionnels 19
7.5 Tests d'acceptation 19
8. ANALYSE DES RISQUES 20
8.1 Erreurs de logique des règles du Go 20
8.2 Problèmes d'interface utilisateur 20
8.3 Performance et redimensionnement 20
8.4 Perte ou corruption des sauvegardes XML 20
8.5 Mauvaises manipulations utilisateur 20
9. APPROBATIONS 20

1. PERSPECTIVE GLOBALE

1.1 Rappel de fonction

L'application permet à deux utilisateurs de jouer au jeu classique GO sur un un seul ordinateur en mulitjoueur local. Particuliers souhaitant jouer au jeu de GO de manière simple, rapide et intuitive, sans nécessiter de connaissances techniques avancées.

Les joueurs peuvent :

- Entrer leurs noms
- Créer une partie
- Jouer sur un plateau de GO
- Passer leur tour
- Abandonner une partie
- Voir le résultat final (score, gagnant, temps)

Le système applique automatiquement :

- Les règles de placement
- La capture des pierres
- La règle du Ko simple
- La gestion des tours
- La fin de partie

1.2 Contraintes

- Support tour par tour (2 utilisateurs).
- Enregistrement local (write-only) en XML.
- L'application doit être simple et intuitive pour tout groupe de personne désirant jouer au jeu
- Application WPF

1.3 Définition de l'architecture de l'application

J'utiliserai l'architecture ***MVVM***, afin d'assurer une séparation claire :

- Model : Instancier/appliquer la logique métier et définition d'objets (Exemples : objet Pierre, Mouvement, Capture…, Planche)
- ViewModel : Liaison et validation des données pour ensuite joindre le Model au View
- View : Interface utilisateur

De plus, j'utiliserai différentes techniques afin de favoriser l'unification et éviter la répétition d'objets. Ainsi, la conception de l'application sera orientée objet, suivra les principes d'encapsulation, d'héritage, de composition et de polymorphisme.

2. MODELISATION ORIENTEE OBJET

2.1 Encapsulation

Les propriétés critiques sont protégées par des méthodes métier empêchant toute modification directe incohérente, afin de centraliser et sécuriser l'application.

Par exemple, le plateau de jeu ne pourra être modifié par d'autre moyen que par une seule méthode robuste, afin d'éviter plusieurs modifications simultanées, et de par le fait même perdre le suivi des modifications.

2.2 Collections et listes d'objets

Certaines listes auront une accessibilité en GET presqu'entièrement globale puisque celles-ci sont nécessaires au fonctionnement de l'application. Par exemple, le tableau de pierres placées et la liste de captures (si nécessaire) devra être partagée dans la plupart des différentes sections pour afficher, modifier ou encore instancier les détails d'un mouvement, d'une attaque ou autre.

- Par exemple, le plateau de jeu contiendra une collection d'états prédéfinis; soit Pierre noire, pierre blanche, ou alors vide/null.

2.3 Héritage

Par le principe de base de POO, l'héritage est priorisé de sorte que tout objet existant peut avoir un objet parent pour partager les propriétés entre objets différents. De même façon, tous les ViewModel héritent de BaseViewModel, qui à son tour implémente INotifyPropertyChanged.

2.4 Rôles par classe

Game et Board :
Gestion de la partie, des tours, fins de jeux, initialise le score.
Board représente le plateau de jeu; sa taille, les positions, les validités.
Player:
Joueur ayant une couleur, nom et un temps pour connaître les deux joueurs adverses.
Move:
Représente un coup joué (par qui, où, quand).
ScoreCalculator:
Calcule le score final selon le joueur, le score et le temps.
XMLService:
Enregistre les résultats dans un fichier XML.

2.5 Classes MODEL

Player

Attributs

Type	Nom
String	Name
Enum	Color
Double	TimePlayed
Move	LastMove
Bool	HasPassed

Méthodes

Nom	Paramètres d'entrée
Constructeur	Name,color
PassTurn	
UpdateTimePlayed	*Time : seconds

Game

Attributs

Type	Nom
Player[…]	Players
Board	Board
Player	Current
Move	LastMove
Double	Komi
Handicap[2]	Handicaps
Bool	isFinished
Bool	isAbandon

Méthodes

Nom	Paramètres d'entrée
Constructeur	
StartGame	
SwitchTurn	
PassTurn	
AbandonGame	
EndGame	
StartGame	

Board

Attributs

Type	Nom
Enum	Size
Stone[][]	Playground
Move	LastMove
Bool	CanPlay
Score?	FinalScore

Méthodes

Nom	Paramètres d'entrée
Constructeur	Size
PlayMove	move
CheckLiberties	x,y
CaptureStones	
IsValid	Move
CheckKo	move
GetStone	x,y

Move

Attributs

Type	Nom
Player	Player
Int[][]	Pos(X,Y)
Double	TimeStamp

Méthodes

Nom	Paramètres d'entrée
Constructeur	Player,x,y

Stone

Attributs

Type	Nom
Player	Parent
Color	Color
Double	TimeStamp

Méthodes

Nom	Paramètres d'entrée
Constructeur	Player, x,y
GetColor	Parent.Color

Score

Attributs

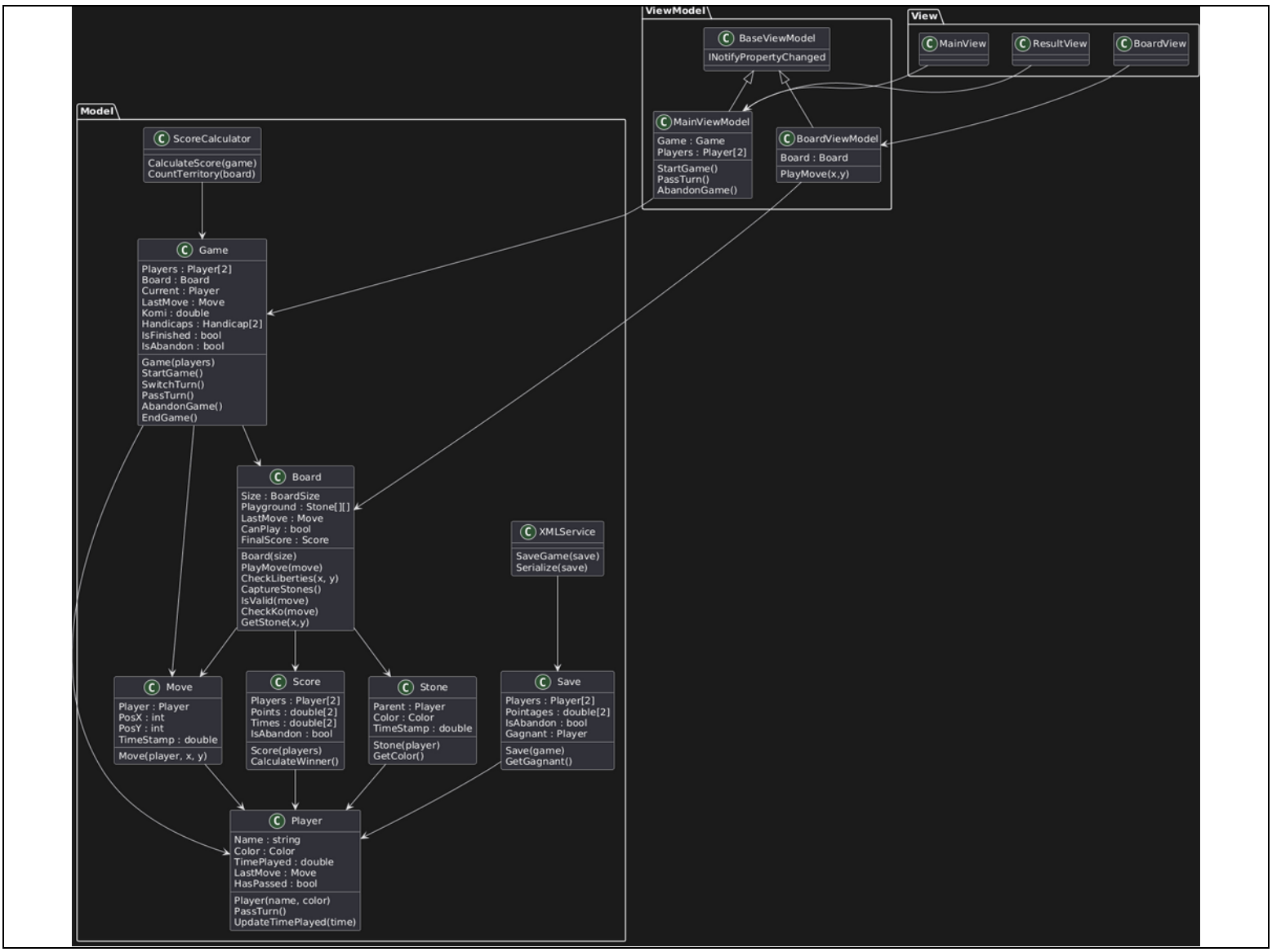
Type	Nom
Player[2]	Players
Double[2]	Points
Double[2]	Times
Bool	isAbandon

Méthodes

Nom	Paramètres d'entrée
Constructeur	
CalculateWinner	

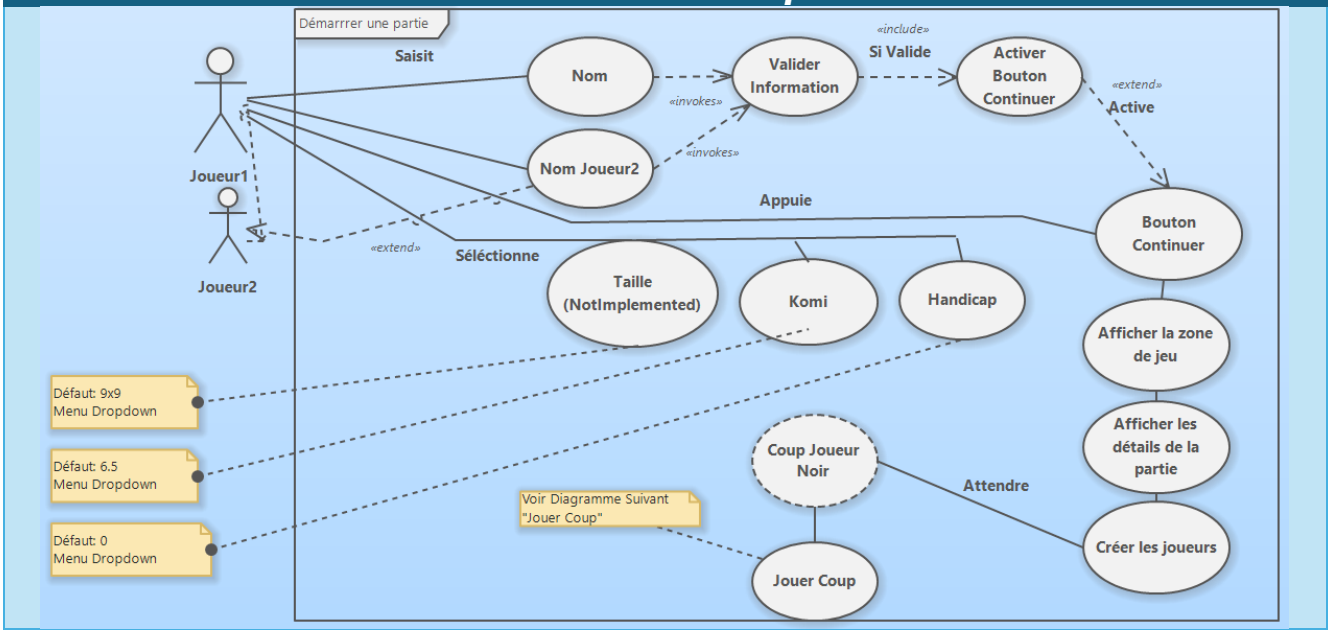
Save			
Attributs		Méthodes	
Type	**Nom**	**Nom**	**Paramètres d'entrée**
Player[2]	Players	**Constructeur**	Game
Double [2]	Pointages	GetGagnant	Players, Pointages, IsAbandon
Bool	isAbandon	CountTerritory	Board
Player	Gagnant		

2.6 Diagramme de classes

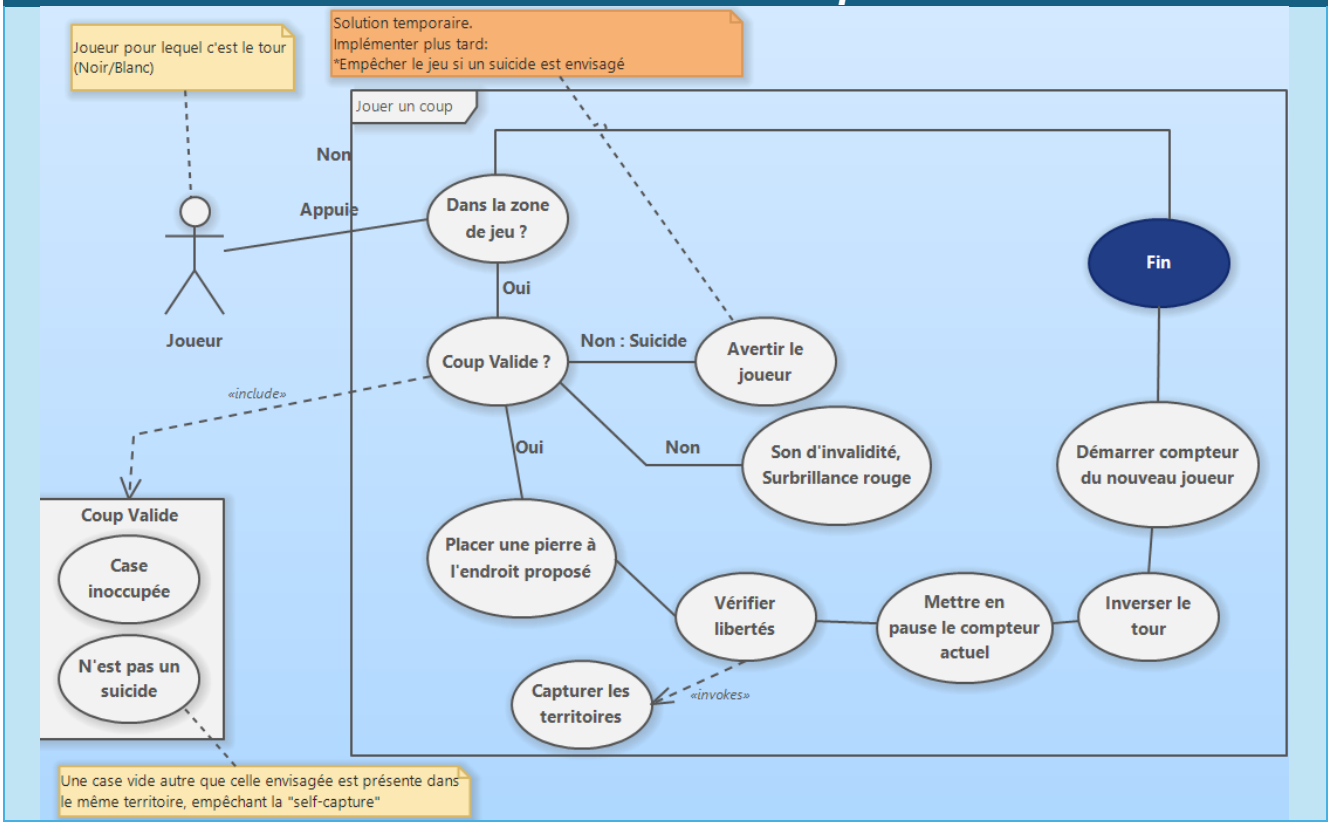


2.7 Diagrammes de cas d'utilisation

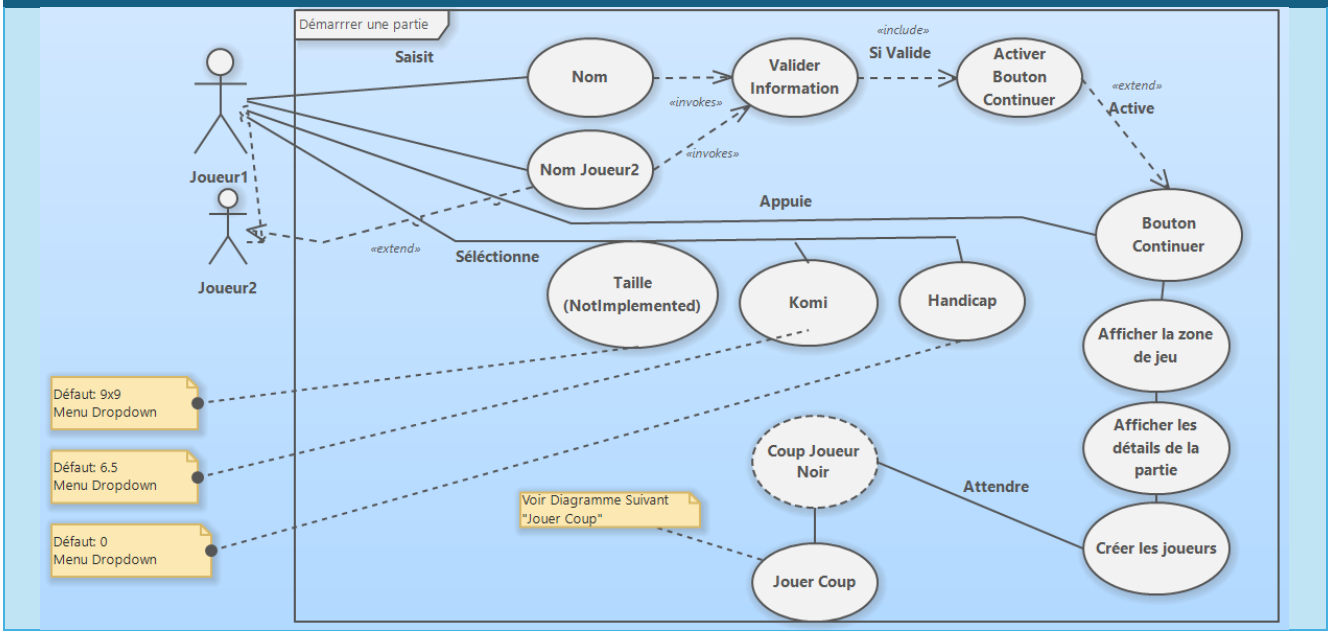
2.7.1 Démarrer la partie



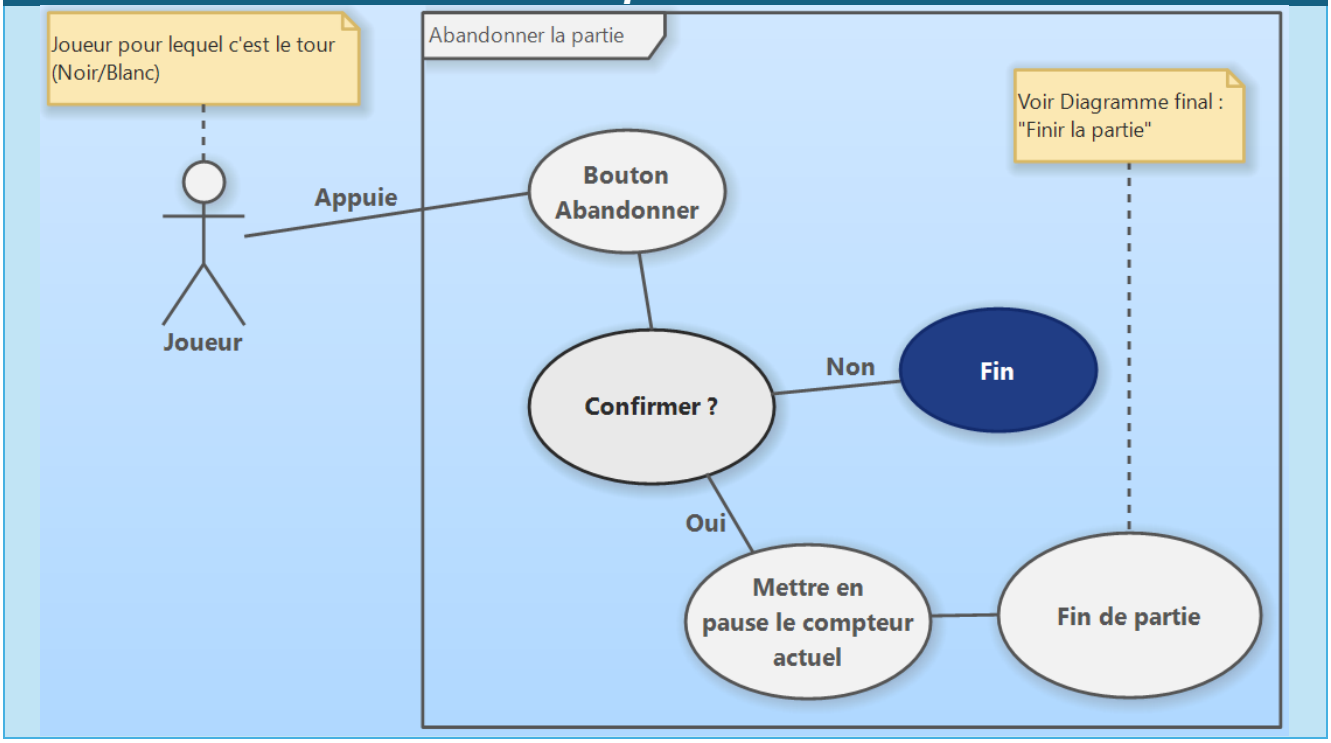
2.7.2 Jouer un coup



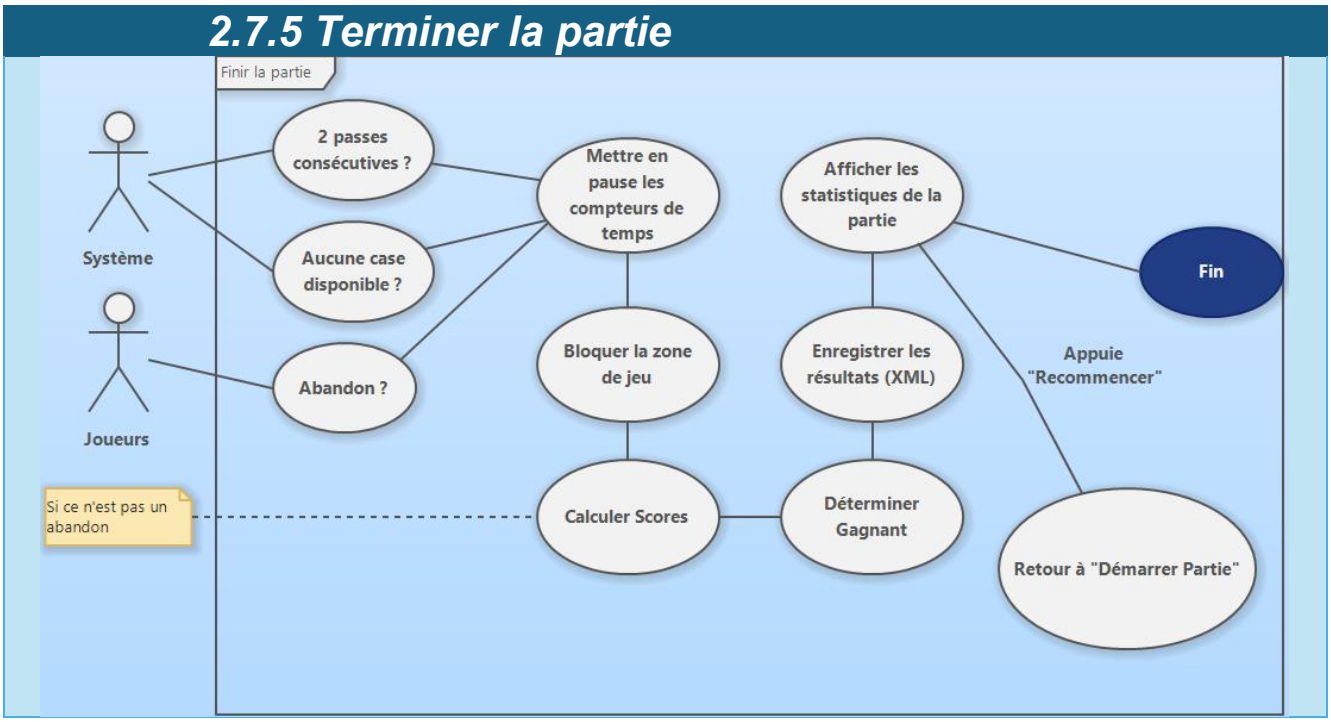
2.7.3 Passer son tour



2.7.4 Abandonner la partie



2.7.5 Terminer la partie



2.8 Validation et rappel de contraintes importantes

- Il est facile de créer une boucle Ko ou une logique circulaire. Il faudra veiller à éviter ce genre de problématique.
- Une case qui n'est pas libre doit automatiquement être disabled ou indisponible. Jouer un coup sur une case occupée entraînerait des problèmes majeurs.
- Chaque coup joué doit instancier une vérification des libertés, et le retrait des groupes capturés dans le cas échéant.
- Les tours doivent toujours être alternés à chaque coup joué : Soit Noir,Blanc,Noir, etc.

4. STRATÉGIE MATHÉMATIQUE

4.1 Détection des libertés

Une pierre possède des libertés si une case adjacente est libre. Ainsi, la vérification des libertés doit se faire pour toutes les cases voisines de celle à vérifier, et continuer par méthode intrinsèque à celles qui avoisinent celles vérifiées, jusqu'à ce qu'une liberté soit trouvée, ou qu'aucune case n'ait de liberté.

Dans un tel cas, le placement d'une pierre serait impossible, car elle menerait à une capture automatique, soit un « suicide ».

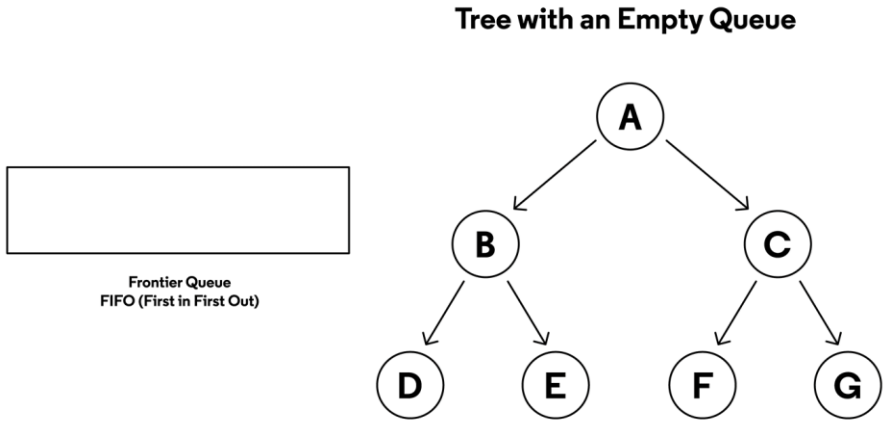
4.1.1 Algorithme de vérification[1]

L'algorithme de vérification des libertés sera basé sur une file d'attente (queue) en mémoire. D'abord, l'algorithme observe les chemins directs possibles à partir d'une pierre donnée, et les ajoutera à la file d'attente.

Ensuite, il vérifie un des éléments de la queue et l'enlève de celle-ci. Si celui-ci correspond, il quitte la vérification plus tôt en affirmant qu'au moins une liberté est présente.

À l'inverse, si l'élément vérifié ne correspond pas, il ajoute les chemins qui s'y connectent qui n'ont pas encore été vérifiés à la queue. Il vérifie ensuite l'élément précédent qui n'a pas encore été vérifié, et recommencera cette étape jusqu'à ce que la queue soit vide.

En suivant cette logique, l'algorithme pourra trouver toutes les libertés si elles sont présentes.



[1] https://www.codecademy.com/article/breadth-first-search-bfs-algorithm

4.2 Pseudo-code

Step 1. **Start from the source node**: Choose a starting node to begin traversal.
Step 2. **Mark the source as visited**: Keep a record of visited nodes to avoid repetition or loops.
Step 3. **Enqueue the source node**: Use a queue (FIFO structure) to store nodes to be explored.
Step 4. **Process the queue**: BFS dequeues a node, visits its unvisited neighbors, marks them visited, and enqueues them.
Step 5. **Move level by level**: Continue until the queue is empty, visiting all nodes layer by layer. [1]

Où :
(x,y) = position du clic
Joueur = noir/blanc selon le tour
V = case voisine de (x,y)

```
Début JouerCoup(x, y, joueur)
{
        CaseOccupee(x, y) ?? return false
        PlacerPierreTemporaire(x, y, joueur)

        Foreach  V(x, y)
                Si PierreAdverse(V) && LibertésGroupe(V) = 0
                        CapturerGroupe(V)

Si LibertésGroupe(x, y) = 0 && aucune capture
        AnnulerPlacement(x, y) Retourner Faux

Si PositionIdentiqueAuTourPrécédent()
        AnnulerPlacement(x, y) Retourner Faux

Finally :
        EnregistrerHistorique() ChangerJoueur() Retourner Vrai
}
```

5. EXIGENCES DE CONCEPTION

Exigence	*Élément à concevoir*
Respect des règles du Go	Validation des coups dans Model
Gestion des tours	Alternance automatique via Game
Détection des captures	Algorithme de libertés + suppression groupes
Règle du Ko	Historique des positions précédentes
Fin de partie	Deux passes consécutives ou abandon
Sauvegarde locale	Export XML des résultats
Interface intuitive	Plateau interactif WPF

6. CONCEPTION DE L'INTERFACE

6.1 Principes de conception

L'interface sera conçue selon les principes de clarté, cohérence et efficacité. Chaque fonctionnalité sera accessible en maximum 3 clics. Les actions fréquentes (détails, passer, abandonner) bénéficieront de raccourcis visuels et seront visibles en permanence. Les couleurs noir/blanc permettront d'identifier facilement les joueurs. Considérant ces indices d'utilisation, l'utilisateur devrait comprendre l'état de la partie sans assistance externe.

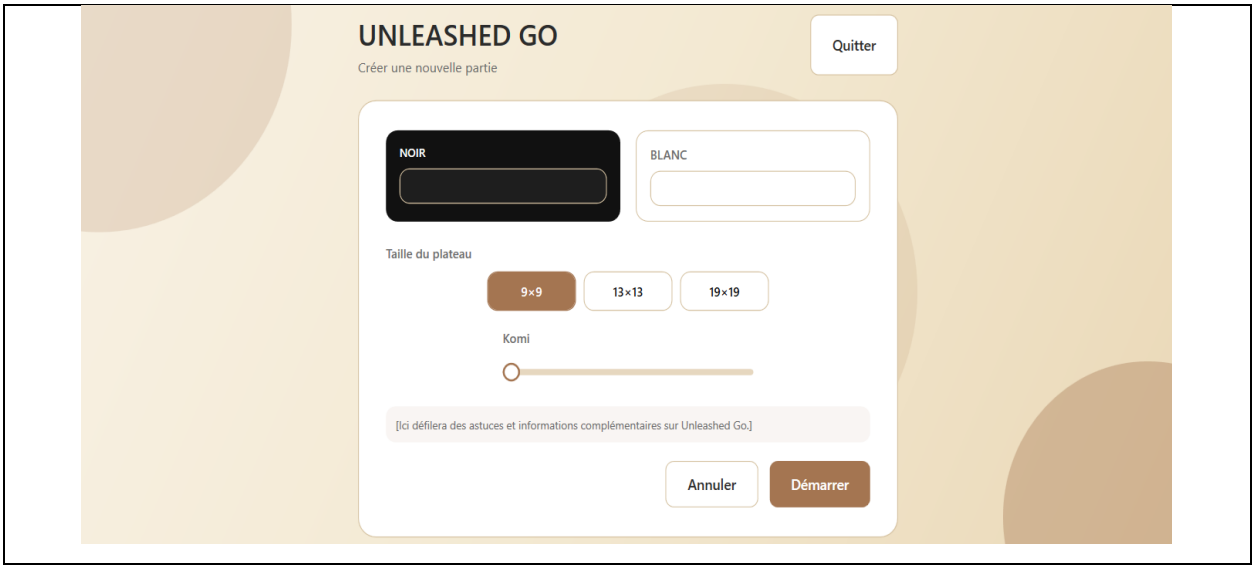
6.2 Architecture de l'interface

La fenêtre principale sera divisée en plusieurs zones fonctionnelles :

- Zone centrale : Plateau de Go interactif;
- Zone droite : Informations de partie (joueur actuel, temps, captures, score estimé);
- Barre supérieure : Nouvelle partie, paramètres, sauvegarde;
- Barre inférieure : Messages système et validations (avertissements, etc);
- Zone latérale optionnelle : Dernier coup joué.

6.3 Maquettes fonctionnelles

6.3.1 Interface de création de partie

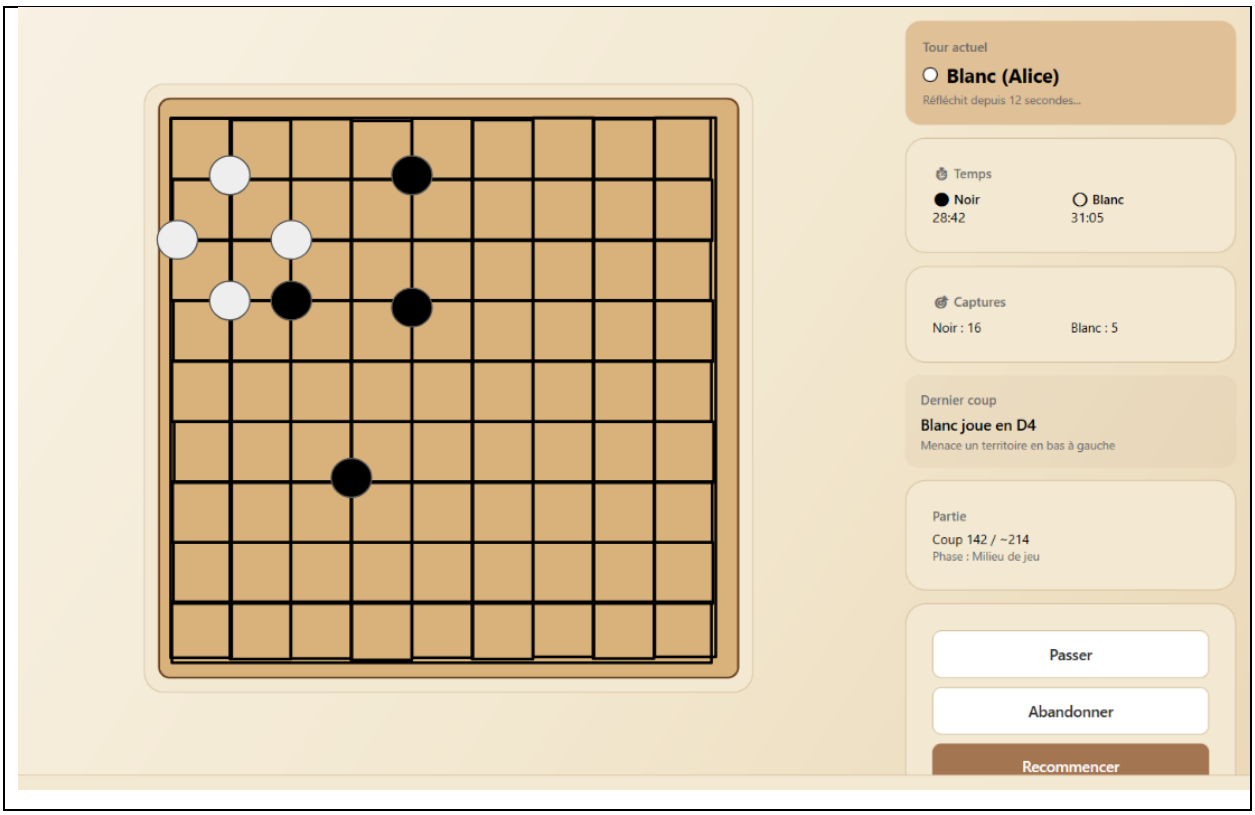


Cet écran apparaît au lancement de l'application.

Éléments affichés :

- Champ Nom du joueur Noir;
- Champ Nom du joueur Blanc;
- Sélection de la taille du plateau : 9x9, 13x13 ou 19x19. (Seul 9x9 est disponible);
- Dropdown Komi avec 6.5 par défaut;
- Bouton Démarrer la partie;
- Bouton Charger une sauvegarde locale;
- Bouton Quitter (annuler).

6.3.2 Interface de jeu en cours



Cet écran représente la section centrale de l'application. Il restera affiché durant toute la partie.

Éléments affichés :

- Plateau de Go interactif occupant la majorité de l'espace.
- Indication claire du joueur dont c'est le tour.
- Chronomètre ou temps total des deux joueurs.
- Nombre de pierres capturées par chaque joueur.
- Dernier coup joué.
- Bouton Passer le tour.
- Bouton Abandonner la partie.
- Bouton Recommencer.
- Barre d'état affichant les messages système : coup invalide, Ko, capture, fin imminente, etc.

6.3.3 Interface de résultats



Cet écran apparaît à la fin de la partie après deux passes consécutives ou un abandon.

Éléments affichés :

- Nom du gagnant.
- Pointage final des deux joueurs.
- Résumé des captures.
- Temps total joué par chaque joueur.
- Cause de fin de partie : double passe ou abandon.
- Bouton Nouvelle partie.
- Bouton Rejouer avec mêmes paramètres.
- Bouton Quitter.
- Bouton Exporter les résultats en XML (Non implémenté).

6.4 Responsivité et redimensionnement

L'interface sera conçue pour s'adapter dynamiquement au redimensionnement de la fenêtre. Les panneaux utiliseront des Grid et StackPanel WPF avec propriétés Width="*" et Height="*". En mode fenêtre réduite, les panneaux secondaires pourront être masqués ou transformés en onglets. Une taille minimale de fenêtre sera définie pour garantir la lisibilité.

7. STRATÉGIES DE TEST

7.1 Philosophie de test

Une stratégie de test complète et rigoureuse sera appliquée pour garantir la qualité du logiciel. Les tests couvriront tous les niveaux : unitaire, intégration, fonctionnel et acceptation, en vérifiant la logique métier, les règles du Go, l'interface et la persistance d'écriture en fichier XML. L'objectif est d'atteindre au minimum 80% de couverture de code pour les couches métier et données.

7.2 Tests unitaires

7.2.1 Objectifs

Valider le comportement individuel de chaque méthode et classe en isolation. Assurer la robustesse des validations, calculs et logiques métier.

7.2.2 Éléments testés

- Classe Board : placement valide / invalide.
- Classe Game : changement de tour.
- Détection des libertés.
- Capture de groupes simples et multiples.
- Validation de la règle du Ko.
- Calcul du score final.
- Sauvegarde XML.

7.3 Tests d'intégration

Valider la communication correcte entre les différents composants du système. S'assurer que les données fonctionnent de bout en bout. Valider les interactions entre ViewModel, Model et View. Vérifier qu'un clic utilisateur produit un coup valide et met l'interface à jour correctement.

7.4 Tests fonctionnels

Valider que l'application répond aux exigences fonctionnelles définies dans le cahier des charges. Tester depuis l'interface utilisateur les fonctionnalités complètes. Valider les scénarios complets : démarrer une partie, jouer plusieurs coups, capturer, passer, abandonner, terminer et sauvegarder.

7.5 Tests d'acceptation

Ces tests seront effectués en présence du client :

- Partie complète sur plateau 9x9;
- Vérification d'une capture connue;
- Vérification d'un Ko simple;
- Double passe menant à la fin de partie;
- Utilisateur capable de lancer une partie sans aide technique (intuitivité);
- Redimensionnement sur différentes résolutions.

8. ANALYSE DES RISQUES

8.1 Erreurs de logique des règles du Go

Un mauvais traitement des règles (captures, suicide, Ko, alternance des tours) pourrait rendre la partie invalide ou frustrante pour les utilisateurs. Ce risque sera réduit par des tests unitaires complets et plusieurs scénarios réels de parties connues.

8.2 Problèmes d'interface utilisateur

Si le plateau manque de clarté visuelle ou si les informations importantes sont difficiles à repérer, les joueurs pourraient commettre des erreurs involontaires. L'interface devra donc rester simple, lisible et centrée sur le plateau.

8.3 Performance et redimensionnement

Sur certains appareils, un plateau 19x19 ou un redimensionnement fréquent pourrait ralentir l'affichage. Il faudra optimiser le rendu graphique et limiter les recalculs inutiles de l'interface.

8.4 Perte ou corruption des sauvegardes XML

Une fermeture inattendue ou une erreur d'écriture pourrait endommager un fichier de sauvegarde. Des validations de fichier et une gestion d'erreurs devront être prévues avant le chargement.

8.5 Mauvaises manipulations utilisateur

Un clic involontaire, un abandon accidentel ou une fermeture rapide de la fenêtre pourrait nuire à l'expérience utilisateur. Les actions critiques devront demander une confirmation préalable.

9. APPROBATIONS

Client Signature : _____ Date : _____	Chef de projet Signature : _____ Date : _____